

Reviewer Pack — Minimal L^p Norm of Affine Line Arrangements Bounds Resoluti...

Entry dd7975f71ce4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 01:35:26 UTC

Minimal L^p Norm of Affine Line Arrangements Bounds Resolution Proof Size

Entry ID: dd7975f71ce4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 01:35:26 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Affine Line Arrangements) **Field B** (complexity object): Boolean Function Complexity: Resolution Proof Complexity

Statement:

For every CNF φ with n variables, the minimal L^p norm of any affine line arrangement that intersects all clauses of φ is upper-bounded by a polynomial in $\log(n)$ and the resolution proof size $t^*(\varphi)$. Specifically, there exists a constant C such that for all $n \leq 40$, $\|L(\varphi)\|_p \leq C * \log(n) * t^*(\varphi)^{(1/p)}$, where $L(\varphi)$ is an affine line arrangement intersecting all clauses of φ .

Rationale (proposer's reasoning):

Affine line arrangements provide a geometric interpretation of clause intersections in CNFs, which could potentially reveal hidden structures related to proof size. The L^p norm measures the complexity of the arrangement, and bounding it by the resolution proof size might expose a connection between geometric and algebraic complexities that can lead to new insights into the proof complexity of SAT.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5b8ee49471a9016a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a CNF φ with n variables ($n \leq 40$), if $\|L(\varphi)\|_p$ is less than or equal to $C * \log(n) * t^*(\varphi)^{(1/p)}$ for all p between 1 and 2, and this holds true for at least 80% of the seeds (24 out of 30).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'affine line arrangements' AND 'resolution proof complexity' AND ' L^p norm' - 'Boolean function complexity' AND 'algebraic geometry' AND 'resolution proof size' - 'CNF formula' AND 'affine line arrangement' AND 'polynomial bound on L^p norm'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(2**n):
            clause = [random.randint(-1, 1) * (i + 1) for i in range(n)]
            if any(x == 0 for x in clause):
                continue
            cnf.append(clause)
        return cnf

    def resolution_proof_size(cnf):
        clauses = set(tuple(sorted(c)) for c in cnf)
        resolvents = []
        while True:
            new_resolvent = False
            for i in range(len(resolvents)):
                for j in range(i + 1, len(resolvents)):
                    common = [x for x in resolvents[i] if -x in resolvents[j]]
                    if common:
                        new_clause = sorted(set(resolvents[i]) | set(resolvents[j]) - {common[0]})
                        if new_clause not in clauses and new_clause not in resolvents:
                            resolvents.append(new_clause)
                            new_resolvent = True
            if not new_resolvent:
                break
```

```

    return len(resolvents)

def l_p_norm(arrangement, p):
    norm = 0
    for line in arrangement:
        norm += sum(abs(x) ** p for x in line) ** (1 / p)
    return norm

n = random.randint(5, 40)
cnf = generate_cnf(n)
t_star = resolution_proof_size(cnf)

if t_star == 0:
    return {
        "metric_name": "L_p_norm",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "resolution_proof_size_is_zero"
    }

arrangement = []
for clause in cnf:
    line = [random.uniform(-1, 1) for _ in range(n + 1)]
    if all(line[i] * (x if x > 0 else -x) >= 0 for x in clause):
        continue
    arrangement.append(line)

if not arrangement:
    return {
        "metric_name": "L_p_norm",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "no_valid_arrangement_found"
    }

metric_value = l_p_norm(arrangement, random.choice([1, 2]))
return {
    "metric_name": "L_p_norm",
    "metric_value": metric_value,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")

```

```

results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"first_failing_seed\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the allotted time frame. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17552
2	preregistration	ollama_remote	glm4:latest	0	0	10720
3	novelty	ollama_remote	glm4:latest	0	0	8642
4	novelty	ollama_remote	glm4:latest	0	0	17982
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18791
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9045
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18557
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12791
9	judge	ollama_remote	glm4:latest	0	0	11558

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 125635 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/dd7975f71ce4.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/dd7975f71ce4.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/dd7975f71ce4.tar.gz (if generated)